

Advanced Algorithms: Homework 7 Solutions

Due on Mar. 27, 2024 at 11:59pm EST

Professor Dana Randall Spring 2024

As stated in the syllabus, unauthorized use
of previous semester course materials is
strictly prohibited in this course.

Sarthak Mohanty

Exercise 1

It is sometimes of interest to compute the Voronoi diagram of a set of sites, but we are only interested in a portion of the final diagram. In this problem, we'll consider how to compute the Voronoi diagram of a set of points in $\mathbb{R}^2$, but restricted to a given line ℓ . By rotating and translating space, we may assume that ℓ is aligned with the x-axis.

You are given a sequence of n sites in the plane $P = \{p_1, \dots, p_n\}$ sorted in increasing order of their x-coordinates (see Fig. 1(a)). Present an algorithm that computes the Voronoi diagram of P , but restricted only to the x-axis. (We don't care about the portion of the diagram lying above or below the axis.)

Observe that the diagram is a sequence of intervals that subdivide the x-axis. The output consists of a sequence of (at most $n - 1$) endpoints of the segments $\{x_1, \dots, x_m\}$, and each edge is labeled with the index of the associated site corresponding to this interval (see Fig. 1(b)). Your algorithm should run in $O(n)$ time. (*Hint*: Start by proving that the left-to-right order of the labels along the x-axis is consistent with the left-to-right order of the sites.)

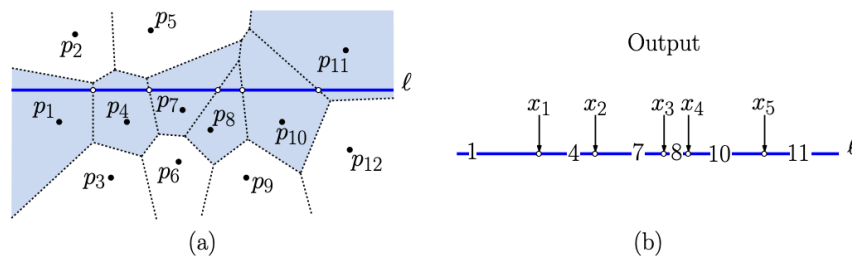


Figure 1: Restriction of a Voronoi diagram to a line.

Solution

First, observe that the left-to-right order of the labels along the x -axis increases monotonically. Indeed, for any two sites $p_i = (x_i, y_i)$ and $p_j = (x_j, y_j)$ with $x_i < x_j$, the point on the x -axis where they are equally close is given by

$$\sqrt{(x - x_i)^2 + y_i^2} = \sqrt{(x - x_j)^2 + y_j^2} \implies x = \frac{x_i^2 - x_j^2 + y_j^2 - y_i^2}{2(x_j - x_i)}.$$

Thus, once p_j becomes closer than p_i on the x -axis, p_i can never become closer than p_j again. Hence the labels along the x -axis are consistent with the left-to-right order of the sites.

Now the key property that we will use is that the distance from a point $(x, 0)$ on the x -axis to a site $p_i = (x_i, y_i)$ is

$$d_i(x)^2 = (x - x_i)^2 + y_i^2.$$

So computing the restricted Voronoi diagram is equivalent to computing the lower envelope of these functions. Since the labels change monotonically, we can compute this lower envelope using a stack.

Initialize a stack with p_1 , and store that p_1 is closest starting from $-\infty$. Now process the points $p_2, \dots, p_n$ from left to right. When inserting a new point p_i , compare it with the current top point p_j on the stack, and compute the point x_{ij} where they are equally close:

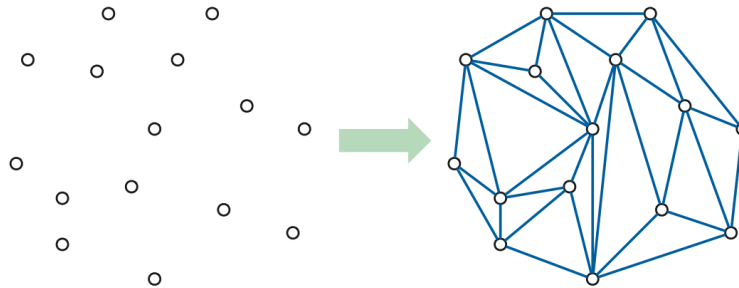
$$x_{ij} = \frac{x_j^2 - x_i^2 + y_i^2 - y_j^2}{2(x_i - x_j)}.$$

If x_{ij} lies to the left of the starting point of p_j 's interval, then p_j never appears on the lower envelope, so pop p_j from the stack and repeat. Otherwise, push p_i onto the stack and record that its interval starts at x_{ij} .

At the end, the stack contains exactly the sites whose Voronoi cells intersect the x -axis, in left-to-right order, together with the endpoints where the label changes. Each point is pushed once and popped at most once, so the total running time is $O(n)$.

Exercise 2

(Erickson) Let P be any set of points in the plane. A triangulation of P is a planar straight-line graph whose outer face is the complement of the convex hull of P , and whose bounded faces are all triangles.



There are several different ways to measure the *quality* of a triangulation. The goal of this question is to prove that the Delaunay triangulation of P is the best possible triangulation of P , for a few different definitions of “best”.

For example, Lecture 17 of [David Mount's notes](#)¹ contains a proof that among all triangulations of P , the Delaunay triangulation of P has the largest minimum angle.

- Let T be any triangulation of P . Prove that if every interior angle of T is acute, then T is the Delaunay triangulation of P .
- For any triangle Δ with vertices pqr , define

$$\text{Vol}(\Delta) = \text{area}(\Delta) \cdot (\|p\|^2 + \|q\|^2 + \|r\|^2)$$

where $\|(a, b)\|^2 = a^2 + b^2$ is the squared Euclidean norm. For any triangulation T , let $\text{Vol}(T) = \sum_{\Delta \in T} \text{Vol}(\Delta)$. Prove that among all triangulations of P , the Delaunay triangulation of P minimizes $\text{Vol}(T)$. [Hint: Why is the function called “Vol”?]

Solution

- Solution 1:** Suppose every interior angle of T is acute, but T is not a Delaunay triangulation. Since T is not Delaunay, there is some triangle pqr in T whose circumcircle contains another point $s \in P$. Among all such choices, choose s so that the segment from s to the triangle pqr crosses the fewest edges of T . If s is not separated from pqr by any edge, then s lies inside pqr , which is impossible since T is a triangulation of P . Thus the segment from s to pqr crosses some edge of T . Let pr be the first such edge, and let prs be the triangle of T on the other side of pr . Then pqr and prs are adjacent triangles, and the common edge pr is not locally Delaunay, since s lies inside the circumcircle of pqr .

¹I gave this as part of your lecture notes.

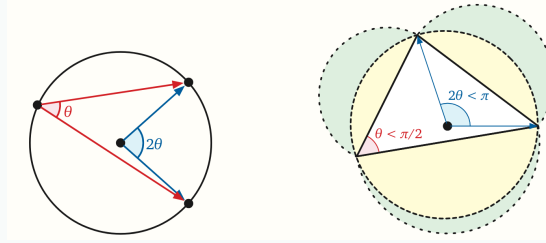
Since s lies inside the circumcircle of pqr , the angle $\angle psr$ is strictly larger than the angle it would make on the boundary of the circle. By the Inscribed Angle Theorem, this boundary angle is supplementary to $\angle pqr$. Therefore,

$$\angle psr + \angle pqr > \pi.$$

Hence at least one of $\angle psr$ and $\angle pqr$ is obtuse, contradicting the assumption that every interior angle of T is acute. Thus T is a Delaunay triangulation.

Solution 2 (Erickson): The *diametral circle* of a line segment is the circle centered at the midpoint of the segment that passes through both endpoints of the segment—more concisely, the smallest circle containing the segment.

We need the following observation, which goes back to Euclid. For any three points p, q, r on a circle with center c , the angle $\angle pcr$ is exactly twice the angle $\angle pqr$. See the figure on the left below.



This observation implies two useful facts about acute triangles:

- For any acute triangle Δ , each vertex of Δ is outside the diametral circle of the opposite edge of Δ .
- The circumcircle of any acute triangle Δ lies inside the union of Δ and the interiors of the diametral circles of the edges of Δ . (See the right figure above.)

These two facts imply that if T is an acute triangulation of P , the diametral circle of every edge of T is empty, and thus the circumcircle of every triangle in T is empty.

- (b) **Solution 1:** Suppose T is not a Delaunay triangulation. Then there must be four points $a, b, c, d \in P$, such that abc and acd are triangles in T whose common edge ac is not locally Delaunay. Suppose a, b, c are oriented counterclockwise. Since ac is not locally Delaunay, d lies inside the circumcircle of $\triangle abc$. Let T' be the triangulation obtained by flipping ac to bd .

The change in $\text{Vol}(T)$ after we perform the edge flip is

$$\begin{aligned} \text{Vol}(T) - \text{Vol}(T') &= (\|a\|^2 + \|b\|^2 + \|c\|^2)\text{area}(abc) + (\|a\|^2 + \|c\|^2 + \|d\|^2)\text{area}(acd) \\ &\quad - (\|a\|^2 + \|b\|^2 + \|d\|^2)\text{area}(abd) - (\|b\|^2 + \|c\|^2 + \|d\|^2)\text{area}(bcd) \\ &= -\|a\|^2\text{area}(bcd) + \|b\|^2\text{area}(acd) - \|c\|^2\text{area}(abd) + \|d\|^2\text{area}(abc). \end{aligned}$$

Now consider the incircle test associated with these four points.

$$\text{inCircle}(a, b, c, d) = \det \begin{pmatrix} a_x & a_y & a_x^2 + a_y^2 & 1 \\ b_x & b_y & b_x^2 + b_y^2 & 1 \\ c_x & c_y & c_x^2 + c_y^2 & 1 \\ d_x & d_y & d_x^2 + d_y^2 & 1 \end{pmatrix}.$$

Observe that a cofactor expansion along the 3rd column yields

$$\text{inCircle}(a, b, c, d) = 2(\text{Vol}(T) - \text{Vol}(T')).$$

Furthermore, d lies inside the circumcircle of $\triangle abc$, so $\text{inCircle}(a, b, c, d) > 0$. Thus $\text{Vol}(T) > \text{Vol}(T')$, so performing the edge flip decreases $\text{Vol}(T)$. Therefore, the Delaunay triangulation minimizes $\text{Vol}(T)$.

Solution 2 (Erickson): For any point p in the plane, let $\hat{p} = (a, b, a^2 + b^2)$ denote its lifting to the unit paraboloid; note that $\|p\|^2 = a^2 + b^2$ is the z -coordinate of $\hat{p}$. For any triangle Δ with vertices pqr , let $\hat{\Delta}$ denote the planar triangle with vertices $\hat{p}, \hat{q}, \hat{r}$. Finally, for any triangulation $T = \{\Delta_1, \Delta_2, \dots\}$ of P , let $\hat{T}$ denote the lifted triangulation $T = \{\hat{\Delta}_1, \hat{\Delta}_2, \dots\}$.

Now consider the vertical triangular prism with bottom facet Δ and top facet $\hat{\Delta}$; the volume of this prism is equal to

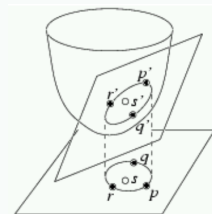
$$\text{area}(\Delta) \cdot \left(\frac{\hat{p}_z + \hat{q}_z + \hat{r}_z}{3} \right) = \frac{\text{Vol}(\Delta)}{3}$$

Thus, $\text{Vol}(T)$ is exactly three times the volume between the lifted triangulation $\hat{T}$ and the plane $z = 0$.

Now let T be a non-Delaunay triangulation of P ; we need to argue that there is another triangulation T' such that $\text{Vol}(T') < \text{Vol}(T)$. Because T is not Delaunay, there must be four points $p, q, r, s \in P$, such that pqr and prs are triangles in T whose common edge pr is not locally Delaunay (basically we can flip it and reduce the angle). Let T' be the triangulation obtained by flipping pr , replacing it with qs . Because pr is not locally Delaunay, this flip lowers the lifted triangulation.

Specifically, we have

$$\frac{\text{Vol}(T')}{3} = \frac{\text{Vol}(T)}{3} - \text{volume}(\hat{p}\hat{q}\hat{r}\hat{s}) < \frac{\text{Vol}(T)}{3}.$$



Exercise 3

Consider the following algorithm for Vertex Cover: Run DFS. Output the nodes which are not leaves in the DFS tree. Show that the output is indeed a vertex cover, and that this algorithm gives yet another 2-approximation for the minimum vertex cover.

Solution

We assume the graph is connected (otherwise, repeat the argument for each connected component). Let T be the DFS tree, and L the set of leaves.

Correctness: First, we show $V \setminus L$ is a vertex cover. Recall $V \setminus L$ is a vertex cover if there are no edges between elements of L in G . Assume there is an edge $(l_1, l_2) \in E$, where $l_1, l_2 \in L$. Then when running the DFS algorithm, when reaching one of those leaves, we could have kept going and traversed that edge - thus one of those leaves is not actually a leaf. This is a contradiction - thus no such edge exists, and $V \setminus L$ is a vertex cover.

2-approx: Any vertex cover of G must also be a vertex cover of $T \subseteq G$, so $\text{OPT}_T \leq \text{OPT}_G$. We consider the problem of finding an optimal vertex cover of T . Note that WLOG, we may assume that no leaves are in the optimal vertex cover: for any leaf that we cover, we may as well cover the parent instead. Also, at least

one of the leaf and the parent must be covered. Thus OPT on a tree covers all parents of leaves. We can then remove all these nodes, and cover the remaining graph iteratively. Let $L = P_0$ be the leaves of T , P_1 be the parents of P_0 , P_2 be the parents of P_1 that are not themselves in P_1 , ..., P_i are the parents of P_{i-1} that are not in any previous P_j , etc. Let M be the maximum such index such that P_M is covered by OPT . Either P_M or P_{M+1} contains the last few vertices of the tree. Set $P_{M+1} = 0$ if the above enumeration ends at P_M . We observe that $P_0 \geq P_1 \geq P_2 \geq \dots \geq P_M \geq P_{M+1}$. We see that OPT_T costs $P_1 + P_3 + P_5 + \dots + P_M$, while our algorithm costs

$$P_1 + P_2 + P_3 + \dots + P_M + P_{M+1} \leq P_1 + P_1 + P_3 + P_3 + \dots + P_M + P_M = 2\text{OPT}_T \leq 2\text{OPT}_G.$$

Exercise 4

We claimed in class that a bad approach to approximating the optimal vertex cover is to use a greedy method that does the following: pick the vertex that covers the most yet uncovered vertices, add it to the cover, and repeat. Give an infinite family of examples (i.e., one for each value of n) that shows that this method will not achieve an approximation ratio of 2.

Solution

There are many possible solutions - in fact, we show that you can't get any constant ratio with the greedy algorithm. For the homework, a counter example for which the greedy algorithm has approximation ratio greater than 2 is sufficient. Let l be a power of two, $l = 2^k$, for convenience.

Let G be a bipartite graph with l vertices on one side, call those vertices A . We construct the other side B iteratively. First add $l/2$ vertices, B_1 , and connect each of these to exactly 2 elements on side A , such that every vertex in A has one neighbor in B_1 . For example, connect b_1 to a_1 and a_2 , then connect b_2 to a_3 and a_4 , etc.

Next, add another $l/2$ vertices, B_2 , and connect each to exactly $2^2 = 4$ vertices on the opposite side, so that each vertex in A will have exactly two neighbors in B_2 . Continue this process until you have set B_k of size $l/2$ which connects to all $2^k = l$ vertices on the other side, such that each vertex in A connects to all $l/2$ of the vertices in B_k .

The degree of a vertex in A is $1 + 2 + 4 + \dots + l/2 = l - 1$. The degree of every node in the vertex set B_k is l , so the greedy algorithm would first select all the nodes of B_k . Now, the number of remaining outgoing uncovered edges for each vertex on side A is $1 + 2 + \dots + l/4 = l/2 - 1$, and the greedy algorithm would then select all the nodes in B_{k-1} which have degree $2^{k-1} = l/2$. Repeating this argument, the greedy algorithm would select all of side B , which has $kl/2$ vertices.

But a better vertex cover would be to choose all of A , which has only l vertices. Thus, we get an approximation factor of $k/2$ - for any k . The total number of vertices $|V| = l + lk/2 = l + l/2 \log l$, so this approximation ratio in terms of $|V|$ is $O(\log|V|) = O(\log l)$.

To get an example for any value of $n = |V|$, even those that can't be written in the form $l + l \log l/2$ for some l , simply find the largest l such that $l + l \log l/2 < n$, and then add isolated vertices until there are n total vertices. This doesn't change either the optimal vertex cover or the greedy vertex cover, since no edges have been added, so we get the same result.

Exercise 5

We saw a version of the load-balancing problem (on the exam) where we had 2 machines. Now we have m machines $M_1 \dots M_m$ and a set of n jobs. Each job j has a processing time t_j . We seek to assign each job to

one of the machines so that the loads placed on all machines are as "balanced" as possible. That is, if $A(i)$ are the set of jobs assigned to machine M_i , then the load for machine M_i is total time required.

$$T_i = \sum_{j \in A(i)} t_j$$

We want to minimize the *makespan* $T = \max_i T_i$. This problem is NP-complete, so instead you should design an approximation algorithm for the problem. What approximation ratio does the greedy algorithm (assign each job to the least loaded machine) yield? Prove your answer.

Solution

Like in the exam, we have two useful bounds for OPT . One is $OPT \geq \frac{1}{m} \sum_{i=1}^n t_i = \frac{1}{m} \sum_{j=1}^m T_j$. This bound says that it is impossible for every machine to have cost lower than this average (otherwise, the total costs of all the machines wouldn't add up to the total weight of all the costs).

Another bound is $OPT \geq \max_i t_i$. This says that the optimal solution has cost at least the largest weighted task (Because this task has to go to *some* machine).

Heuristically, if we have a large number of tasks where every individual task has small weight, the first bound is useful but the second is not. If we have one task of weight much larger than the others, then the second bound is probably more relevant.

We start by looking at the results of ALG , the greedy algorithm. Say it outputs an assignment where each machine i has cost T_i , and WLOG put them in order so that $T_1 \leq T_2 \leq \dots \leq T_m$. Then the cost of $ALG = T_m$. Our first bound says $OPT \geq \frac{1}{m} \sum_{i=1}^m T_i$. Because this is the greedy algorithm, the last element put on the heaviest machine, T_m , must have been placed when machine m had the lowest cost, in particular it must have been lower than T_1 . Thus, this task had cost at least $T_m - T_1$, and we have that $OPT \geq T_m - T_1$. This is just in terms of T_1 and T_m , so let's use a more convenient first bound for OPT ,

$$OPT \geq \frac{1}{m} \sum_{i=1}^m T_i \geq \frac{1}{m} \left(\left(\sum_{i=1}^{m-1} T_1 \right) + T_m \right) = \frac{m-1}{m} T_1 + \frac{1}{m} T_m.$$

We have two cases depending on which of these bounds is larger (and thus the more useful bound).

Say the last task on M_m is the better bound. Then $T_m - T_1 \geq \frac{m-1}{m} T_1 + \frac{1}{m} T_m$. This boils down to $T_1 \leq \frac{m-1}{2m-1} T_m$. We see that

$$\begin{aligned} \frac{ALG}{OPT} &\leq \frac{T_m}{T_m - T_1} \\ &\leq \frac{T_m}{T_m - \frac{m-1}{2m-1} T_m} \\ &= \frac{1}{\frac{m}{2m-1}} = \frac{2m-1}{m} \end{aligned}$$

Say instead that the first bound is the better bound: $\frac{m-1}{m} T_1 + \frac{1}{m} T_m \geq T_m - T_1$. Then $T_1 \geq \frac{m-1}{2m-1} T_m$. Again,

we use the larger one as our bound on OPT .

$$\begin{aligned}\frac{ALG}{OPT} &\leq \frac{T_m}{\frac{m-1}{m}T_1 - \frac{1}{m}T_m} \\ &\leq \frac{T_m}{\frac{m-1}{2m-1} \frac{m-1}{m}T_m - \frac{1}{m}T_m} \\ &= \frac{m(2m-1)}{(m-1)^2 - (2m-1)} \\ &= \frac{m(2m-1)}{(m)^2} = \frac{2m-1}{m}\end{aligned}$$

Thus in each case, we get a bound of $\frac{2m-1}{m} = 2 - \frac{1}{m}$.